TASK 2: DEEP LEARNING PROJECT

Handwritten Digit Classification Using a Convolutional Neural Network

1. Objective

The objective of this project is to develop a deep-learning model that recognizes handwritten digits from 0 to 9. A Convolutional Neural Network (CNN) is trained using the MNIST dataset and evaluated on unseen test images.

2. Tools and Technologies

- Python
- Google Colab
- TensorFlow and Keras
- NumPy
- Matplotlib
- Scikit-learn

3. Dataset Description

The MNIST dataset contains 70,000 grayscale images of handwritten digits. It includes 60,000 training images and 10,000 testing images. Each image has a size of 28 × 28 pixels and belongs to one of ten classes, from 0 to 9.

4. Data Preprocessing

The pixel values were converted to floating-point numbers and normalized from the range 0–255 to 0–1. A channel dimension was added to represent each grayscale image in the format required by the Convolutional Neural Network.

5. Model Architecture

The model uses a Convolutional Neural Network (CNN). It contains two convolutional layers with 32 and 64 filters, followed by max-pooling layers. The extracted features are flattened and passed to a dense layer containing 128 neurons with ReLU activation. A dropout layer with a rate of 0.3 helps reduce overfitting. The output layer contains ten neurons with softmax activation to classify digits from 0 to 9.

6. Training Configuration

The model uses the Adam optimizer and sparse categorical cross-entropy loss. It is trained with a batch size of 128 for up to eight epochs. Ten percent of the training data is used for validation. Early stopping monitors validation loss and restores the best weights when training stops early.

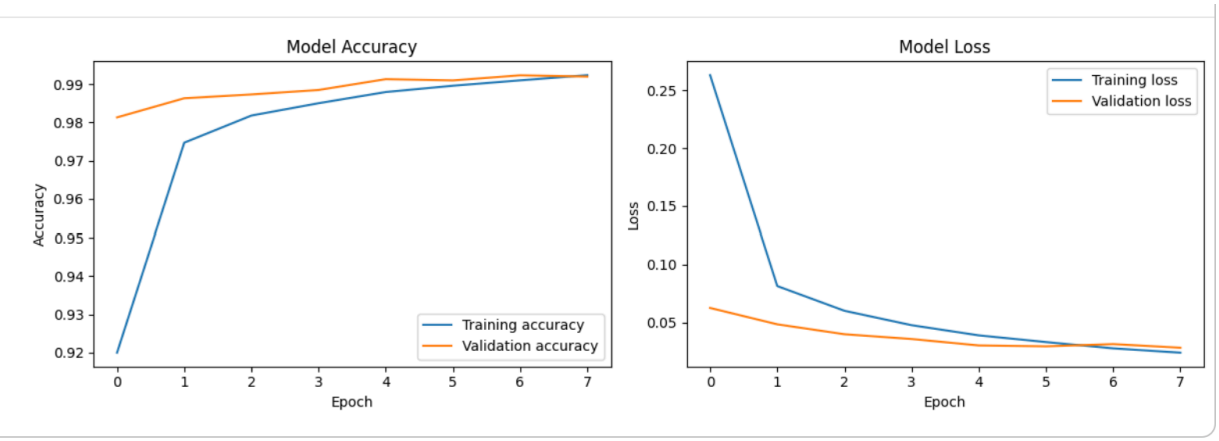
7. Model Evaluation

The model is evaluated using the separate test dataset. The evaluation includes test loss, test accuracy, precision, recall, F1-score, and a confusion matrix. Training and validation accuracy and loss graphs are also generated to understand the learning process.

Actual results from Google Colab:

```
Test loss: 0.0246
Test accuracy: 99.14%
```

```
Epoch 1/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 15s 19ms/step - accuracy: 0.9200 - loss: 0.2629 - val_accuracy: 0.9813
Epoch 2/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9747 - loss: 0.0815 - val_accuracy: 0.9863 -
Epoch 3/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9818 - loss: 0.0602 - val_accuracy: 0.9873 -
Epoch 4/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9850 - loss: 0.0477 - val_accuracy: 0.9885 -
Epoch 5/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9880 - loss: 0.0389 - val_accuracy: 0.9913 -
Epoch 6/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.9896 - loss: 0.0332 - val_accuracy: 0.9910 -
Epoch 7/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9910 - loss: 0.0277 - val_accuracy: 0.9923 -
Epoch 8/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9924 - loss: 0.0240 - val_accuracy: 0.9920 -
Model training completed!
```



8. Results and Discussion

The model learns patterns from handwritten digit images and predicts the corresponding digit. The training and validation graphs show the model's learning progress. The classification report measures performance for each digit, while the confusion matrix shows correct and incorrect predictions.

The final discussion should be based on the actual test accuracy and graphs obtained from the executed notebook.

9. Prediction and Visualization

The trained model predicts digits from test images. The actual and predicted labels are displayed for comparison. The confusion matrix helps identify which digits are classified correctly and which digits are confused with one another.

10. Saving and Loading the Model

The trained model is saved in Keras format as `mnist_model.keras`. The saved model is then loaded again and tested on a sample image to verify that it can be reused for prediction.

11. Conclusion

This project demonstrates an end-to-end deep-learning workflow, including data loading, preprocessing, CNN architecture design, model training, evaluation, visualization, prediction, and model saving. The project shows how deep learning can be applied to handwritten digit classification.

12. GitHub Repository

GitHub repository: [Paste your Task 2 repository link here]

13. Evidence

Include screenshots of the following outputs from Google Colab:

1. Sample MNIST images.
2. CNN model summary.
3. Training progress and final test accuracy.
4. Training and validation accuracy and loss graphs.
5. Sample digit predictions.
6. Classification report and confusion matrix.

```
Epoch 3/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9818 - loss: 0.0602 - val_accuracy: 0.9873 - val_loss: 0.0399
Epoch 4/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9850 - loss: 0.0477 - val_accuracy: 0.9885 - val_loss: 0.0358
Epoch 5/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9880 - loss: 0.0389 - val_accuracy: 0.9913 - val_loss: 0.0302
Epoch 6/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.9896 - loss: 0.0332 - val_accuracy: 0.9910 - val_loss: 0.0295
Epoch 7/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9910 - loss: 0.0277 - val_accuracy: 0.9923 - val_loss: 0.0314
Epoch 8/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9924 - loss: 0.0240 - val_accuracy: 0.9920 - val_loss: 0.0283
Model training completed!
```

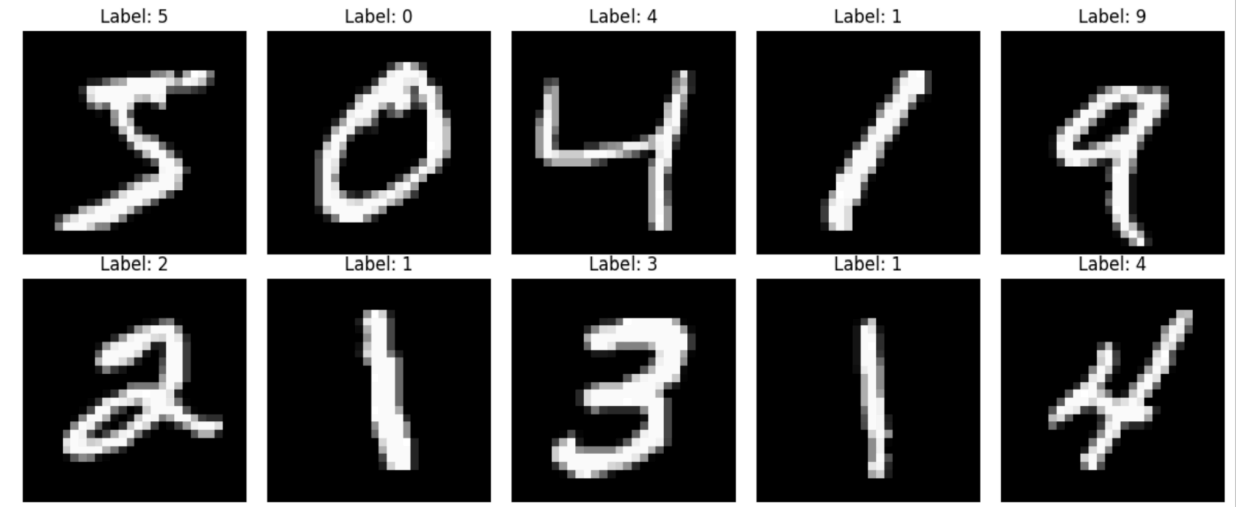
```
# STEP 6: Evaluate on unseen test images

test_loss, test_accuracy = model.evaluate(
    x_test_cnn,
    y_test,
    verbose=0
)

print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")
```

7.

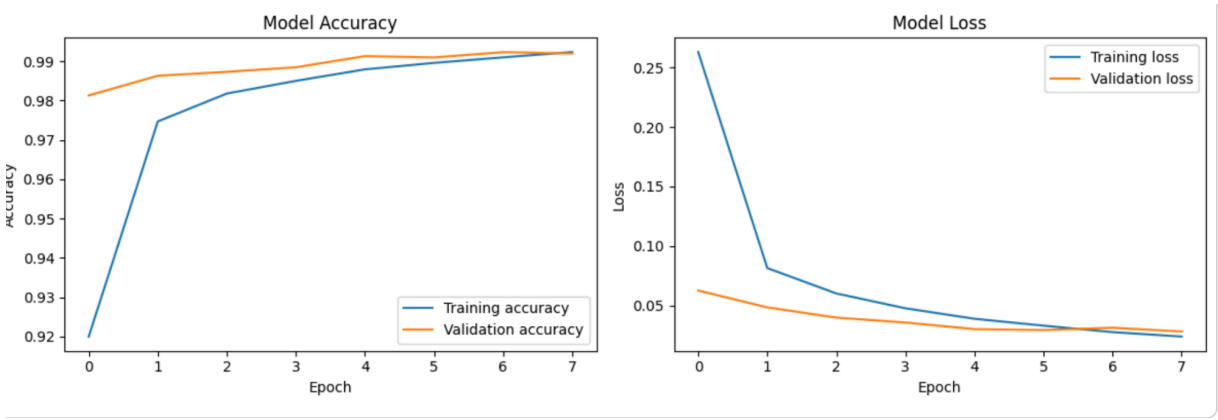
```
Test loss: 0.0246
Test accuracy: 99.14%
```

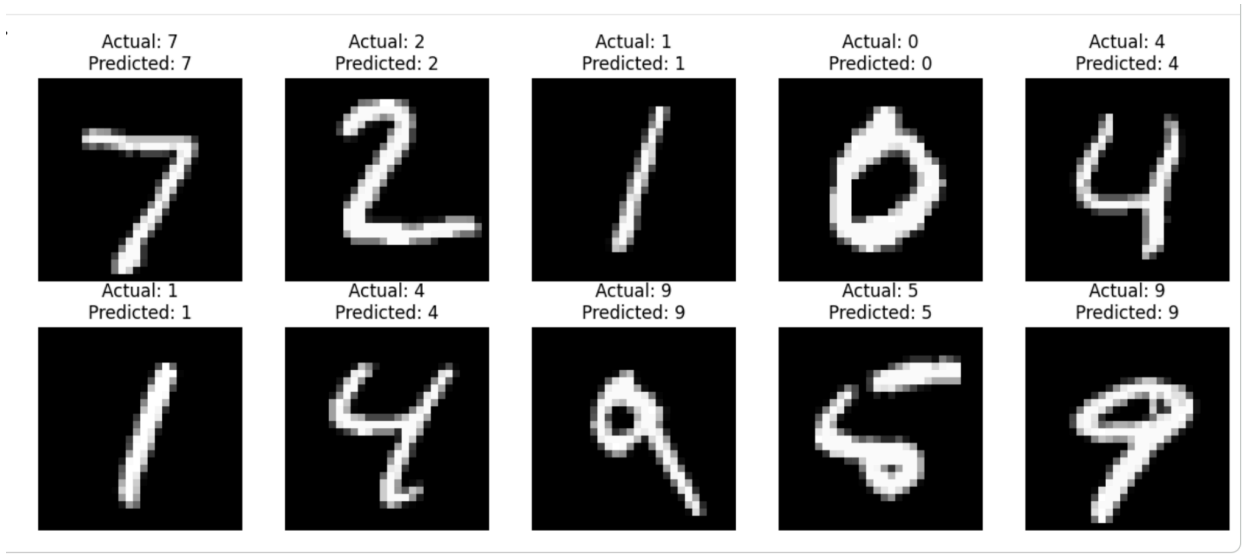


Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)


Model Accuracy
Accuracy
Epoch
Training accuracy
Validation accuracy
Model Loss
Loss
Epoch
Training loss
Validation loss


Actual: 7
Predicted: 7
Actual: 2
Predicted: 2
Actual: 1
Predicted: 1
Actual: 0
Predicted: 0
Actual: 4
Predicted: 4
Actual: 1
Predicted: 1
Actual: 4
Predicted: 4
Actual: 9
Predicted: 9
Actual: 5
Predicted: 5
Actual: 9
Predicted: 9

```
Classification Report:
              precision    recall  f1-score   support

           0     0.9919    0.9969    0.9944       980
           1     0.9965    0.9947    0.9956      1135
           2     0.9961    0.9864    0.9912      1032
           3     0.9901    0.9931    0.9916      1010
           4     0.9929    0.9939    0.9934       982
           5     0.9800    0.9910    0.9855       892
           6     0.9979    0.9843    0.9911       958
           7     0.9874    0.9912    0.9893      1028
           8     0.9908    0.9908    0.9908       974
           9     0.9891    0.9911    0.9901      1009

    accuracy                         0.9914     10000
   macro avg     0.9913    0.9914    0.9913     10000
weighted avg     0.9914    0.9914    0.9914     10000
```

